

Unit – IV

Intermediate SQL (Foreign Keys, Transactions, and ACID)

Part 1: Foreign Keys

What You Will Learn

1. What foreign keys are and what problems they solve.
2. How to set different protection mechanisms (referential actions) for data integrity.

What Problems Can Foreign Keys Solve?

A foreign key (FOREIGN KEY) is a relational binding column (such as `user_id` or `order_id`) placed in a table to reference data in an external table.

While relational data can be maintained manually, foreign key **constraints** instruct the database engine to enforce automatic integrity checks. This prevents critical data corruption—such as accidentally deleting a user record while leaving orphaned order records behind in the system.

Basic Syntax & Usage

Foreign key constraints are defined on a table using the ALTER TABLE statement:

```
ALTER TABLE `new_schema`.`orders`  
ADD CONSTRAINT `orders_user_id_key`  
FOREIGN KEY (`user_id`)  
REFERENCES `new_schema`.`users` (`id`);
```

Syntax Breakdown:

1. ALTER TABLE: Specifies the child table receiving the foreign key restriction (orders).
2. ADD CONSTRAINT: Assigns a unique, descriptive identifier name (orders_user_id_key) to the constraint rule.
3. FOREIGN KEY: Specifies the local column (user_id) bound as the foreign key.
4. REFERENCES: Identifies the parent table and primary key column (users(id)) being linked.

Data Protection Mechanisms

1. Protection from DELETE

If you attempt to delete a referenced parent record:

```
DELETE FROM `new_schema`.`users` WHERE `id` = '1';
```

The database blocks execution and triggers an error:

ERROR 1451: Cannot delete or update a parent row: a foreign key constraint fails.

This occurs because order records depend on `user_id = 1`.

2. Protection from UPDATE

Attempting to update a referenced primary key column:

```
UPDATE `new_schema`.`users` SET `id` = '6' WHERE `id` = '1';
```

Triggers the same **ERROR 1451** because changing the primary key breaks the link to dependent order records.

However, non-relational column updates or reassigning existing foreign key references execute successfully:

-- Allowed: Updating non-key attributes

```
UPDATE `new_schema`.`users` SET `name` = 'Tony' WHERE `id` = '1';
```

-- Allowed: Reassigning an order to a different valid user

```
UPDATE `new_schema`.`orders` SET `user_id` = '3' WHERE `id` = '1';
```

Removing a Constraint

To modify or update an existing constraint, you must first drop the old constraint before adding a new one:

```
ALTER TABLE `new_schema`.`orders`  
DROP FOREIGN KEY `orders_user_id_key`;
```

Referential Actions & Modes

You can explicitly set execution rules for ON DELETE and ON UPDATE actions.

```
ALTER TABLE `new_schema`.`orders`  
ADD CONSTRAINT `orders_user_id_key`  
FOREIGN KEY (`user_id`)  
REFERENCES `new_schema`.`users` (`id`)  
ON DELETE NO ACTION  
ON UPDATE RESTRICT;
```

Common Referential Action Modes:

Mode	Behavior
NO ACTION / RESTRICT	Default. Prevents DELETE or UPDATE on parent tables if child references exist.
CASCADE	Propagates changes automatically. Updating/deleting a parent ID updates or deletes matching child rows.
SET NULL	Automatically sets foreign key columns in child tables to NULL when the parent record is altered or deleted.

Part 2: SQL Transactions

What You Will Learn

1. What database transactions are and why they are critical.
2. Using transactions under normal conditions (COMMIT).
3. Reverting transactions under error conditions (ROLLBACK).

Concept

A **Transaction** groups multiple SQL statements into a single, indivisible logical unit.

Unlike standard single query executions, a transaction holds queries in a temporary state. Based on business logic or error detection, the batch can either be permanently saved to the database (COMMIT) or entirely reversed (ROLLBACK).

Basic Transaction Commands

```
-- Step 1: Start transaction state  
START TRANSACTION;
```

```
-- Step 2: Execute data changes  
UPDATE `new_schema`.`products` SET `price` = '500' WHERE `id` = 5;  
UPDATE `new_schema`.`products` SET `price` = '500' WHERE `id` = 6;
```

```
-- Step 3: Persist all changes permanently  
COMMIT;
```

Note: Many tools (such as MySQL Workbench) run in AUTOCOMMIT mode by default. Using START TRANSACTION explicitly disables autocommit until COMMIT or ROLLBACK is executed.

Error Recovery with ROLLBACK

If an error or failure occurs mid-transaction, ROLLBACK cancels all uncommitted queries and restores data to its original state:

```
START TRANSACTION;
```

```
-- First change executes in transaction state
```

```
UPDATE `new_schema`.`products` SET `price` = '500' WHERE `id` = 5;
```

```
-- An application error occurs here!
```

```
-- Cancel all modifications and revert changes
```

```
ROLLBACK;
```

Programmatic Control Logic

In compound procedures or application layers, transaction status is checked dynamically:

```
START TRANSACTION;
```

```
SELECT `new_schema`.`products` WHERE `id` = 5;
```

```
UPDATE `new_schema`.`products` SET `price` = '500' WHERE `id` = 5;
```

```
IF (@correct) THEN
```

```
    COMMIT;
```

```
ELSE
```

```
    ROLLBACK;
```

```
END IF;
```

Part 3: ACID Principles

What is ACID?

ACID is an acronym representing four core guarantees that relational database management systems enforce to ensure data integrity during transactional operations:

1. Atomicity ("All or Nothing")

- **Definition:** A transaction is treated as a single, atomic unit of work. Every query inside it must either succeed completely or fail completely.
- **Example:** In a money transfer between Account A and Account B, the debit from A and credit to B must happen together. If a crash occurs after debiting A, Atomicity ensures the entire transaction rolls back so A does not lose funds without B receiving them.

2. Consistency ("Data Correctness")

- **Definition:** Ensures the database transitions strictly from one valid state to another, maintaining all predefined schema constraints, foreign keys, and rules.
- **Example:** If a system crash or error occurs mid-transaction, ROLLBACK returns the data to its pre-transaction state without violating integrity constraints.

3. Isolation ("Concurrent Execution")

- **Definition:** Concurrent transactions execute without interfering with one another. Intermediate, uncommitted states inside a transaction remain invisible to other concurrent sessions.
- **Example:** If multiple users attempt to withdraw funds from account A simultaneously, row locking ensures each request queues properly to prevent race conditions or invalid balances.

4. Durability ("Permanent Storage")

- **Definition:** Once a transaction executes COMMIT, its results are permanently written to non-volatile storage and will survive subsequent power losses or system crashes.
- **Example:** If a power outage occurs immediately after a transaction commits, the system recovers with all committed data intact.

SOCIAL MEDIA

Database Design Overview Entities

- Users
- Platforms (Instagram, X, LinkedIn, etc.)
- Posts
- Reactions (Likes, Comments, Shares)
- Followers

Table Creation Script (DDL)

Users Table

```
CREATE TABLE users ( user_id INT PRIMARY KEY, username VARCHAR(50) NOT NULL, join_date DATE, country VARCHAR(50) );
```

Platforms Table

```
CREATE TABLE platforms ( platform_id INT PRIMARY KEY, platform_name VARCHAR(30) UNIQUE NOT NULL );
```

Posts Table

```
CREATE TABLE posts ( post_id INT PRIMARY KEY, user_id INT, platform_id INT, post_date DATE, content_type VARCHAR(20), -- image, video, reel, text CONSTRAINT fk_post_user FOREIGN KEY (user_id) REFERENCES users(user_id), CONSTRAINT fk_post_platform FOREIGN KEY (platform_id) REFERENCES platforms(platform_id) );
```

Reactions Table

```
CREATE TABLE reactions ( reaction_id INT PRIMARY KEY, post_id INT, reaction_type VARCHAR(20), -- like, comment, share reaction_date DATE, CONSTRAINT fk_reaction_post FOREIGN KEY (post_id) REFERENCES posts(post_id) );
```

Followers Table

```
CREATE TABLE followers ( follower_id INT, following_id INT, follow_date DATE, PRIMARY KEY (follower_id, following_id), CONSTRAINT fk_follower FOREIGN KEY (follower_id) REFERENCES users(user_id), CONSTRAINT fk_following FOREIGN KEY (following_id) REFERENCES users(user_id) );
```

Sample Data Insertion (DML)

Users

```
INSERT INTO users VALUES (1, 'arjun_ai', DATE '2024-01-10', 'India');  
INSERT INTO users VALUES (2, 'meera_data', DATE '2024-02-15', 'India');  
INSERT INTO users VALUES (3, 'john_ml', DATE '2023-11-05', 'USA');  
INSERT INTO users VALUES (4, 'sara_cloud', DATE '2024-03-01', 'UK');
```

Platforms

```
INSERT INTO platforms VALUES (1, 'Instagram');  
INSERT INTO platforms VALUES (2, 'X');  
INSERT INTO platforms VALUES (3, 'LinkedIn');
```

Posts

```
INSERT INTO posts VALUES (101, 1, 1, DATE '2024-06-01', 'reel');  
INSERT INTO posts VALUES (102, 2, 1, DATE '2024-06-02', 'image');  
INSERT INTO posts VALUES (103, 3, 2, DATE '2024-06-03', 'text');  
INSERT INTO posts VALUES (104, 1, 3, DATE '2024-06-04', 'article');
```

Reactions

```
INSERT INTO reactions VALUES (1001, 101, 'like', DATE '2024-06-01');  
INSERT INTO reactions VALUES (1002, 101, 'comment', DATE '2024-06-01');  
INSERT INTO reactions VALUES (1003, 101, 'share', DATE '2024-06-02');  
INSERT INTO reactions VALUES (1004, 102, 'like', DATE '2024-06-02');  
INSERT INTO reactions VALUES (1005, 103, 'like', DATE '2024-06-03');  
INSERT INTO reactions VALUES (1006, 104, 'comment', DATE '2024-06-04');
```

Followers

```
INSERT INTO followers VALUES (2, 1, DATE '2024-05-01');  
INSERT INTO followers VALUES (3, 1, DATE '2024-05-02');  
INSERT INTO followers VALUES (4, 2, DATE '2024-05-03');
```

Analytics Queries (Core Learning Part)

1. Total Engagement per Post

```
SELECT p.post_id, COUNT(r.reaction_id) AS total_engagement FROM posts p LEFT JOIN reactions r ON p.post_id = r.post_id GROUP BY p.post_id;
```

	post_id	total_engagement
▶	101	3
	102	1
	103	1
	104	1

2. Engagement Breakdown (Like / Comment / Share)

```
SELECT p.post_id, r.reaction_type, COUNT(*) AS reaction_count FROM posts p JOIN reactions r ON p.post_id = r.post_id GROUP BY p.post_id, r.reaction_type;
```

	post_id	reaction_type	reaction_count
▶	101	like	1
	101	comment	1
	101	share	1
	104	comment	1
	102	like	1
	103	like	1

3. Top 5 Most Engaging Posts

```
SELECT * FROM ( SELECT p.post_id, COUNT(r.reaction_id) AS engagement, RANK() OVER (ORDER BY COUNT(r.reaction_id) DESC) AS rn FROM posts p LEFT JOIN reactions r ON p.post_id = r.post_id GROUP BY p.post_id ) WHERE rn <= 5;
```

	post_id	engagement	rnk
▶	101	3	1
	104	1	2
	102	1	2
	103	1	2

4. Most Engaging Users

```
SELECT u.username, COUNT(r.reaction_id) AS total_engagement FROM users u JOIN posts p  
ON u.user_id = p.user_id LEFT JOIN reactions r ON p.post_id = r.post_id GROUP BY  
u.username ORDER BY total_engagement DESC;
```

	username	total_engagement
▶	arjun_ai	4
	meera_data	1
	john_ml	1

5. Users Who Never Posted

```
SELECT u.username FROM users u LEFT JOIN posts p ON u.user_id = p.user_id WHERE  
p.post_id IS NULL;
```

Result Grid				Filter
	username			
▶	sara_cloud			

6. Average Reactions per Post

```
SELECT AVG(reaction_count) AS avg_reactions FROM ( SELECT post_id, COUNT(*) AS  
reaction_count FROM reactions GROUP BY post_id );
```

Result Grid			
	avg_reactions		
▶	1.5000		

7. Platform-wise Engagement

```
SELECT pl.platform_name, COUNT(r.reaction_id) AS total_engagement FROM platforms pl  
JOIN posts p ON pl.platform_id = p.platform_id LEFT JOIN reactions r ON p.post_id =  
r.post_id GROUP BY pl.platform_name;
```

Result Grid





Filter Rows:

	platform_name	total_engagement
▶	Instagram	4
	LinkedIn	1
	X	1

8. Peak Posting Day

```
SELECT post_date, COUNT(*) AS post_count FROM posts GROUP BY post_date ORDER BY post_count DESC;
```

Result Grid			Filter Rows:
	post_date	post_count	
▶	2024-06-01	1	
	2024-06-02	1	
	2024-06-03	1	
	2024-06-04	1	

9. Users with More Than 1,000 Engagements (Scalable Query)

```
SELECT u.username FROM users u JOIN posts p ON u.user_id = p.user_id JOIN reactions r ON p.post_id = r.post_id GROUP BY u.username HAVING COUNT(r.reaction_id) > 1000;
```

Result Grid		Filter Rows:
	username	

10. Followers Count per User

```
SELECT u.username, COUNT(f.follower_id) AS follower_count FROM users u LEFT JOIN followers f ON u.user_id = f.following_id GROUP BY u.username;
```

Result Grid			Filter Rows:
	username	follower_count	
▶	arjun_ai	2	
	meera_data	1	
	john_ml	0	
	sara_cloud	0	